МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 5

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Шерри Георгий Даниэлевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

ОБЪЕКТЫ БАЗЫ ДАННЫХ: ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ, ХРАНИМЫЕ ПРОЦЕДУРЫ И ТРАНЗАКЦИИ

Цель работы: Работа направлена на формирование у студентов углубленных навыков работы с объектами баз данных в СУБД PostgreSQL, смещая акцент от прямого манипулирования данными к созданию переиспользуемых логических конструкций и безопасному управлению транзакциями.

Ниже приводятся таблицы, используемые для построения запросов.

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	full_name			passport_data			email	login	password
		last_name	first_name	middle_name	series	number	issued_by	issue_date			
1	1	Иванов	Иван	Иванович	4510	123456	УФМС России по г. Москве	2015-03-10	ivanov@mail.ru	ivan_i	pass123
2	2	Петрова	Анна	Сергеевна	5012	789012	МВД по Республике Татарст	2018-11-20	anna.petrova@list	annap	qwerty
3	3	Сидоров	Алексей	Владимирович	4612	345678	УФМС по Ростовской облас	2020-01-25	sid_alex@yandex.r	alexs	1q2w3e
4	4	Козлова	Елена	Дмитриевна	4711	987654	Отдел УФМС по г. Санкт-Пет	2019-06-30	lenakozlova@gma	lenak	zaq12wsx
5	5	Морозов	Дмитрий	Алексеевич	4522	556677	УФМС по Краснодарскому к	2021-02-14	morozov_d@bk.ru	dmorozov	pass321

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	name	country_id	city_id	stars	full_address					
						postal_code	country	city	street	building	apartment
1	1	Sunset Beach Resort	2	3	5	07100	Турция	Анталья	Lara Beach	Bina Yolu No:15	[NULL]
2	2	Grand Whale	1	1	4	141008	Россия	Москва	Ленинградское ш	37	1B
3	3	Jaz Aquamarine	3	5	5	84511	Египет	Хургада	Safaga Road	1	[NULL]
4	4	Green Park Resort	4	8	4	83150	Таиланд	Пхукет	Patong Beach Roa	133	[NULL]
5	5	Atlantis The Palm	5	9	5	00000	ОАЭ	Дубай	Palm Jumeirah	1	[NULL]

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

Введите SQL выражение чтобы отфильтровать результаты

	id	created_at	client_id	employee_id	status_id	paid_at	total_amount
1	1	2025-01-10 10:15:00.000 +0300	1	2	5	2025-01-10 12:00:00.000 +0300	85 000
2	2	2025-01-15 14:30:00.000 +0300	2	4	4	[NULL]	120 000
3	3	2025-01-20 09:45:00.000 +0300	3	5	1	[NULL]	45 000
4	4	2025-02-05 11:20:00.000 +0300	4	6	2	[NULL]	210 000
5	5	2025-02-10 16:00:00.000 +0300	5	2	6	2025-02-11 10:00:00.000 +0300	67 000

client 1 | hotel 1 (2) | customer_order 1 (3) | order_item 1 (4) | Статистика 1

	service_id	order_id	quantity	price
1	1	1	1	85 000
2	3	2	2	60 000
3	5	3	1	45 000
4	2	4	1	210 000
5	4	5	1	67 000

Задание №1: разработка пользовательских функций

Листинг 1

```
CREATE OR REPLACE FUNCTION get_client_total_spent(p_client_id INT)
RETURNS NUMERIC(10,2) AS $$
DECLARE
    total_spent NUMERIC(10,2);
    client_exists BOOLEAN;
BEGIN
    -- Проверка существования клиента
    SELECT EXISTS(SELECT 1 FROM client WHERE id = p_client_id) INTO
client_exists;

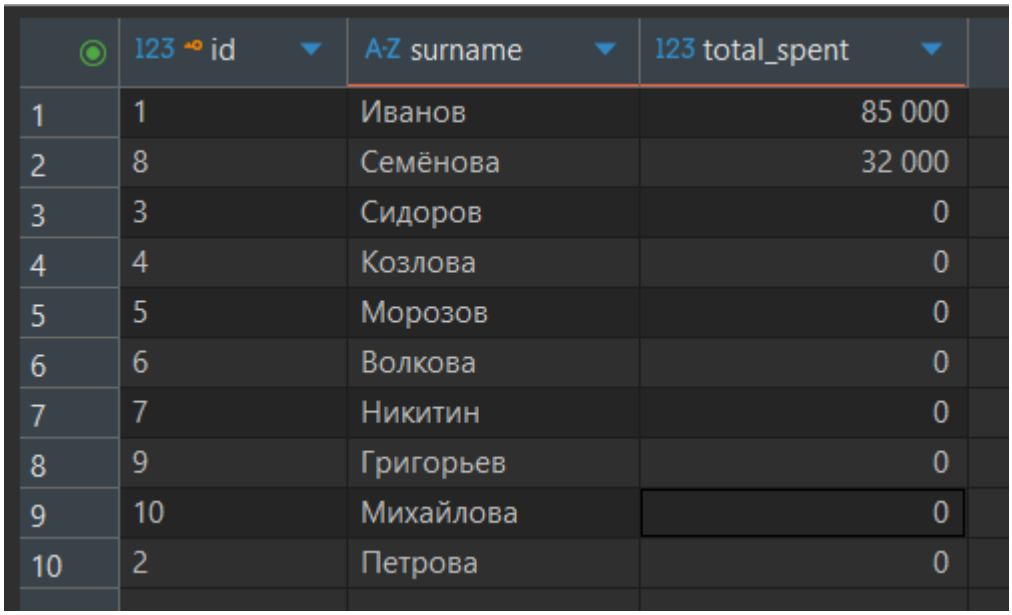
    IF NOT client_exists THEN
        RAISE EXCEPTION 'Клиент с ID % не найден.', p_client_id;
    END IF;

    -- Подсчет суммы (только для заказов со статусом "Оплачен полностью")
    SELECT COALESCE(SUM(total_amount), 0.00) INTO total_spent
    FROM customer_order
    WHERE client_id = p_client_id
      AND status_id = (SELECT id FROM order_status WHERE name = 'Оплачен
полностью');

    RETURN total_spent;
END;
$$ LANGUAGE plpgsql;

-- Вызов функции:
SELECT id, (full_name).last_name AS surname, get_client_total_spent(id) AS
total_spent
FROM client
ORDER BY total_spent DESC;
```

Name	Value
Updated Rows	0
Execute time	0,003s
Start time	Sat Oct 03 19:20:31 MSK 2026
Finish time	Sat Oct 03 19:20:32 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_client_total_spent(p_client_id INT) RETURNS NUMERIC(10,2) AS $$ DECLARE total_spent NUMERIC(10,2); client_exists BOOLEAN; BEGIN -- Проверка существования клиента SELECT EXISTS(SELECT 1 FROM client WHERE id = p_client_id) INTO client_exists; IF NOT client_exists THEN RAISE EXCEPTION 'Клиент с ID % не найден.', p_client_id; END IF; -- Подсчет суммы (только для заказов со статусом "Оплачен полностью") SELECT COALESCE(SUM(total_amount), 0.00) INTO total_spent FROM customer_order WHERE client_id = p_client_id AND status_id = (SELECT id FROM order_status WHERE name = 'Оплачен полностью'); RETURN total_spent; END; $$ LANGUAGE plpgsql



	123 id	A-Z surname	123 total_spent
1	1	Иванов	85 000
2	8	Семёнова	32 000
3	3	Сидоров	0
4	4	Козлова	0
5	5	Морозов	0
6	6	Волкова	0
7	7	Никитин	0
8	9	Григорьев	0
9	10	Михайлова	0
10	2	Петрова	0

Листинг 2

```
CREATE OR REPLACE FUNCTION get_hotel_info(
    p_hotel_id INT,
    OUT h_name VARCHAR,
    OUT h_stars SMALLINT,
    OUT h_country VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    SELECT h.name, h.stars, c.name
    INTO h_name, h_stars, h_country
    FROM hotel h
    JOIN country c ON h.country_id = c.id
    WHERE h.id = p_hotel_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Отель с ID % не найден.', p_hotel_id;
    END IF;
END;
$$;

-- Вызов функции:
SELECT * FROM get_hotel_info(1);
```

Статистика 1

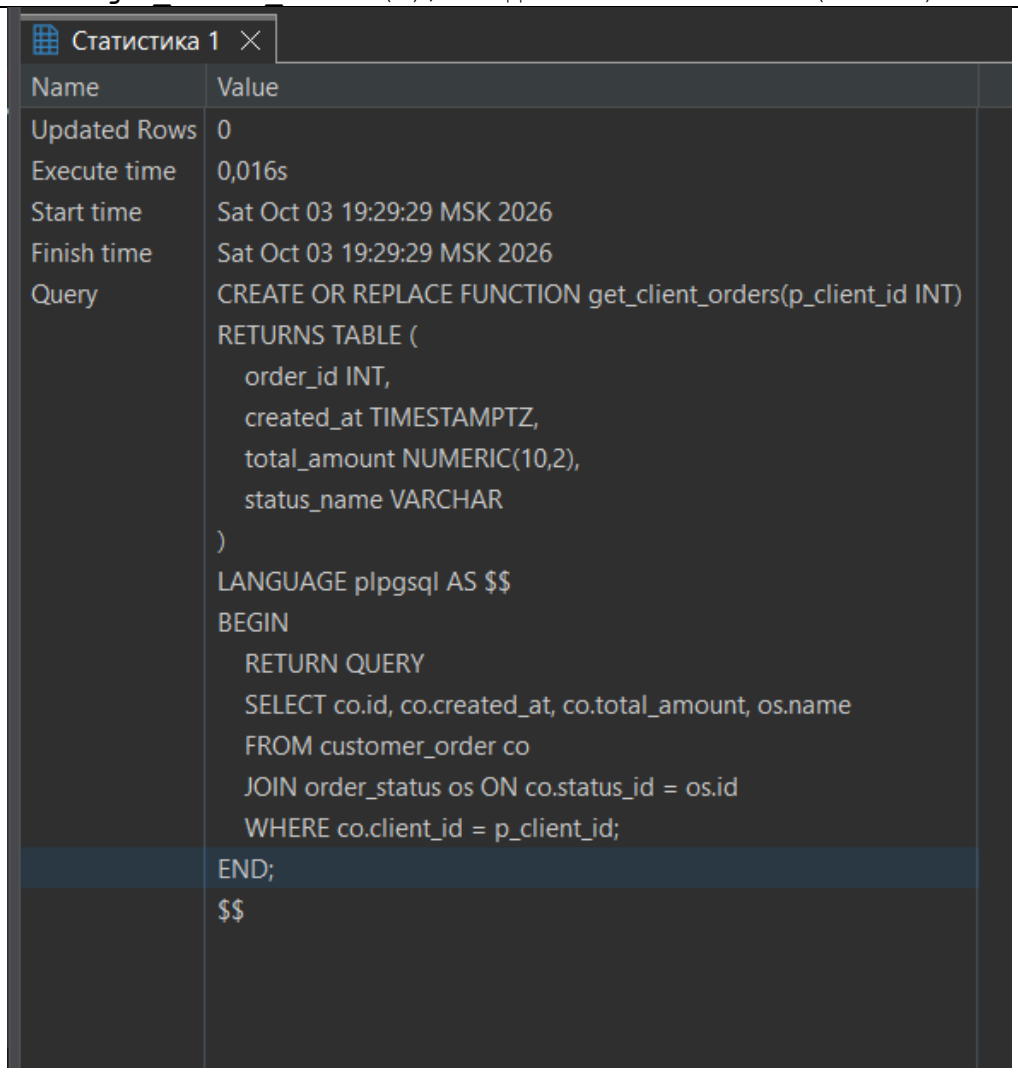
Name	Value
Updated Rows	0
Execute time	0,004s
Start time	Sat Oct 03 19:27:28 MSK 2026
Finish time	Sat Oct 03 19:27:28 MSK 2026
Query	CREATE OR REPLACE FUNCTION get_hotel_info(p_hotel_id INT, OUT h_name VARCHAR, OUT h_stars SMALLINT, OUT h_country VARCHAR) LANGUAGE plpgsql AS $$ BEGIN SELECT h.name, h.stars, c.name INTO h_name, h_stars, h_country FROM hotel h JOIN country c ON h.country_id = c.id WHERE h.id = p_hotel_id; IF NOT FOUND THEN RAISE EXCEPTION 'Отель с ID % не найден.', p_hotel_id; END IF; END; $$

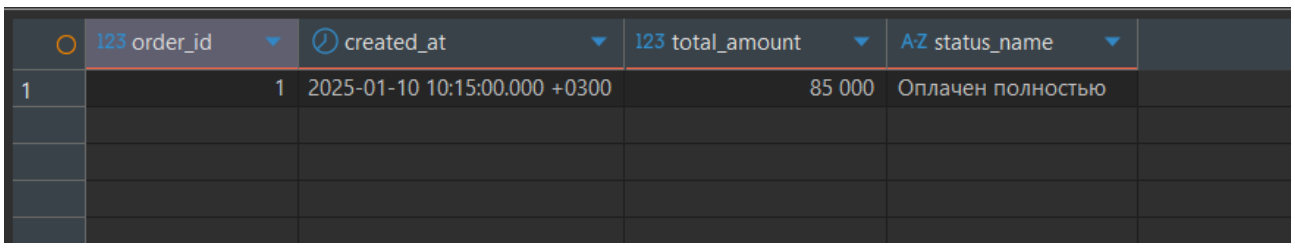
	A-Z h_name	123 h_stars	A-Z h_country
1	Sunset Beach Resort	5	Турция

Листинг 3

```
CREATE OR REPLACE FUNCTION get_client_orders(p_client_id INT)
RETURNS TABLE (
    order_id INT,
    created_at TIMESTAMPTZ,
    total_amount NUMERIC(10,2),
    status_name VARCHAR
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT co.id, co.created_at, co.total_amount, os.name
    FROM customer_order co
    JOIN order_status os ON co.status_id = os.id
    WHERE co.client_id = p_client_id;
END;
$$;

-- Вызов функции:
SELECT * FROM get_client_orders(1); -- Для клиента с id=1 (Иванов)
```





Задание №2: разработка хранимой процедуры с управлением транзакциями

Листинг 4

```
CREATE OR REPLACE PROCEDURE process_order_payment(
    p_order_id INT,
    p_payment_amount NUMERIC(10,2),
    p_payment_type_id INT,
    OUT p_success BOOLEAN,
    OUT p_message TEXT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_order_total NUMERIC(10,2);
    v_order_status INT;
    v_paid_status_id INT;
BEGIN
    -- Получаем ID статуса "Оплачен полностью"
    SELECT id INTO v_paid_status_id FROM order_status WHERE name = 'Оплачен
полностью';

    -- Блокируем строку заказа (FOR UPDATE) для предотвращения состояния гонки
    SELECT total_amount, status_id INTO v_order_total, v_order_status
    FROM customer_order
    WHERE id = p_order_id
    FOR UPDATE;

    -- Проверка 1: Заказ существует?
    IF NOT FOUND THEN
        p_success := FALSE;
        p_message := 'Ошибка: Заказ не найден.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Проверка 2: Заказ не отменен?
    IF v_order_status IN ((SELECT id FROM order_status WHERE name IN ('Отменён
клиентом', 'Отменён поставщиком'))) THEN
        p_success := FALSE;
        p_message := 'Ошибка: Заказ отменен, оплата невозможна.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Проверка 3: Сумма оплаты корректна?
    IF p_payment_amount <= 0 OR p_payment_amount > v_order_total THEN
        p_success := FALSE;
        p_message := 'Ошибка: Некорректная сумма оплаты.';
        ROLLBACK;
        RETURN;
    END IF;

    -- Успешные DML-операции
    INSERT INTO payment (order_id, amount, status_id, payment_type_id)
    VALUES (p_order_id, p_payment_amount, v_paid_status_id,
p_payment_type_id);

    UPDATE customer_order
    SET paid_at = CURRENT_TIMESTAMP,
        status_id = v_paid_status_id
    WHERE id = p_order_id;
```

```
    -- Фиксация транзакции
    COMMIT;
    p_success := TRUE;
    p_message := 'Оплата успешно проведена.';
END;
$$;
```

Name	Value
Updated Rows	0
Execute time	0,006s
Start time	Sat Oct 03 19:37:46 MSK 2026
Finish time	Sat Oct 03 19:37:46 MSK 2026

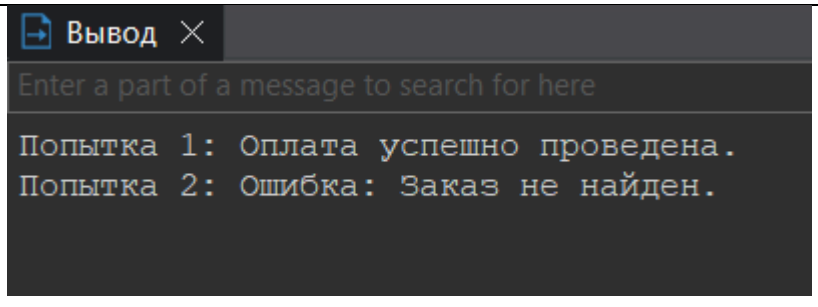
Query	CREATE OR REPLACE PROCEDURE process_order_payment(
	p_order_id INT,
	p_payment_amount NUMERIC(10,2),
	p_payment_type_id INT,
	OUT p_success BOOLEAN,
	OUT p_message TEXT
	)
	LANGUAGE plpgsql AS $$
	DECLARE
	v_order_total NUMERIC(10,2);
	v_order_status INT;
	v_paid_status_id INT;
	BEGIN
	-- Получаем ID статуса "Оплачен полностью"
	SELECT id INTO v_paid_status_id FROM order_status WHERE name = 'Оплачен полностью';
	-- Блокируем строку заказа (FOR UPDATE) для предотвращения состояния гонки
	SELECT total_amount, status_id INTO v_order_total, v_order_status
	FROM customer_order
	WHERE id = p_order_id
	FOR UPDATE;
	-- Проверка 1: Заказ существует?
	IF NOT FOUND THEN
	p_success := FALSE;

Демонстрация вызова (успешный и неудачный):

Листинг 5

```
DO $$
DECLARE
    v_success BOOLEAN;
    v_message TEXT;
BEGIN
    -- Попытка 1: Успешная оплата (Заказ id=3, сумма 45000, тип оплаты СБП
id=3)
    CALL process_order_payment(3, 45000.00, 3, v_success, v_message);
    RAISE NOTICE 'Попытка 1: %', v_message;

    -- Попытка 2: Неудачная (Заказ не существует)
    CALL process_order_payment(9999, 1000.00, 1, v_success, v_message);
    RAISE NOTICE 'Попытка 2: %', v_message;
END;
$$;
```



Задание №3: демонстрация атомарной транзакции (COMMIT)

Обновить email клиента и добавить ему новый телефон. Обе операции должны выполниться вместе.

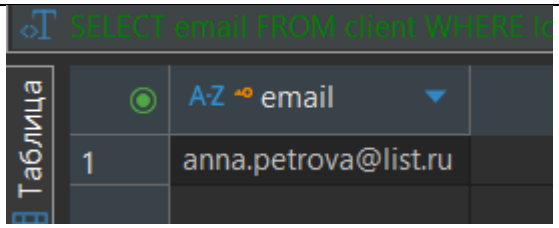
Листинг 6

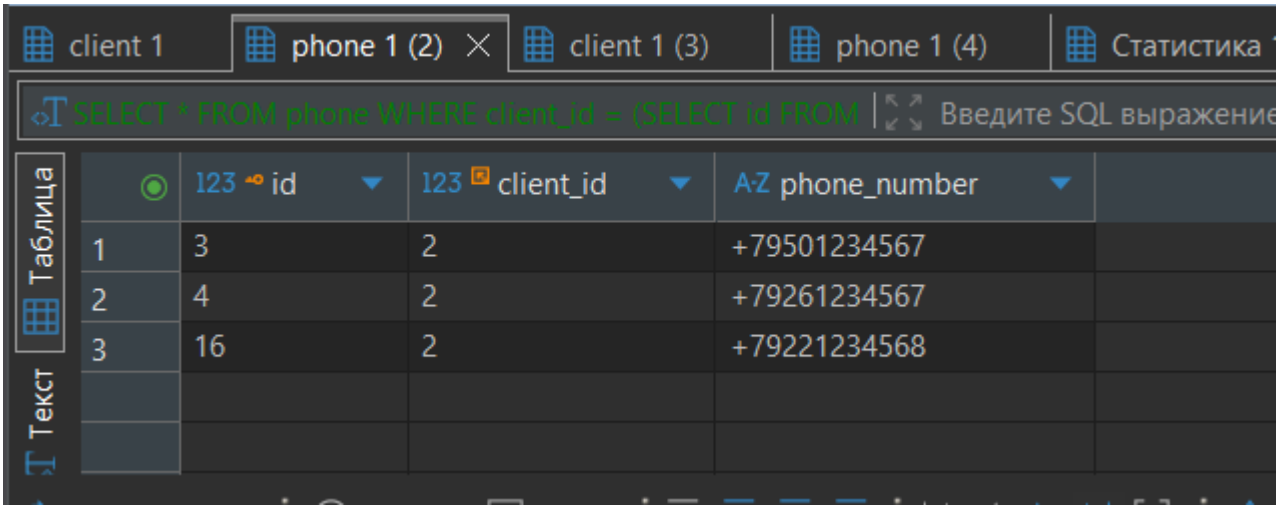
```
-- Шаг 1. Анализ "ДО"
SELECT email FROM client WHERE login = 'annap';
SELECT * FROM phone WHERE client_id = (SELECT id FROM client WHERE login =
'annap');

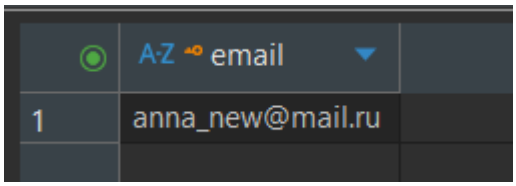
-- Шаг 2. Выполняем атомарную операцию
BEGIN;
    UPDATE client
    SET email = 'anna_new@mail.ru'
    WHERE login = 'annap';

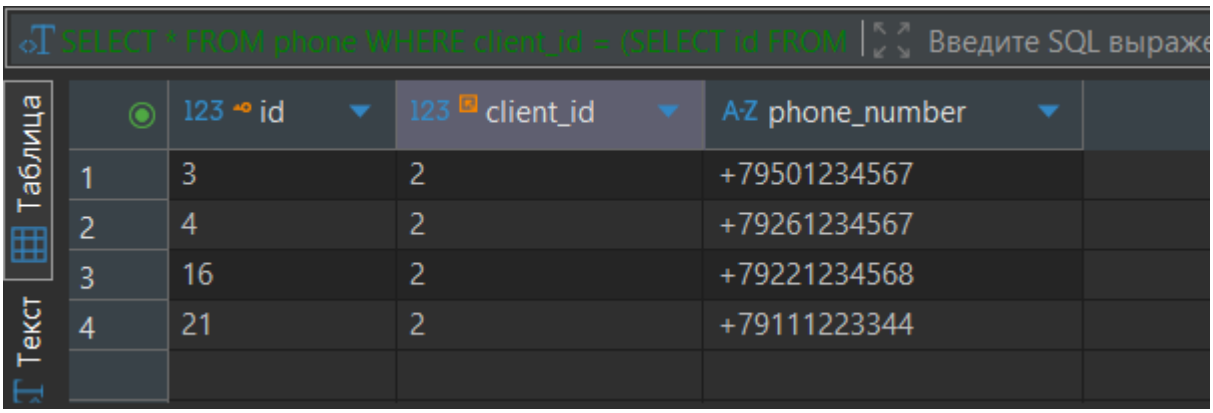
    INSERT INTO phone (client_id, phone_number)
    VALUES ((SELECT id FROM client WHERE login = 'annap'), '+79111223344');
COMMIT;

-- Шаг 3. Анализ "ПОСЛЕ"
SELECT email FROM client WHERE login = 'annap';
SELECT * FROM phone WHERE client_id = (SELECT id FROM client WHERE login =
'annap');
```









Задание №4: демонстрация отката транзакции (ROLLBACK)

Листинг 7

```
-- Ошибка возникнет из-за того, что id = 1 уже существует в таблице country
(это Россия).
BEGIN;
    -- 1. Успешное обновление (звезды отеля изменятся, но пока только внутри
транзакции)
    UPDATE hotel SET stars = 5 WHERE id = 10;

    -- 2. Намеренная ошибка: нарушение PRIMARY KEY (id=1 уже есть)
    INSERT INTO country (id, name, code)
    VALUES (1, 'Тестовая страна', 'TST');

COMMIT;

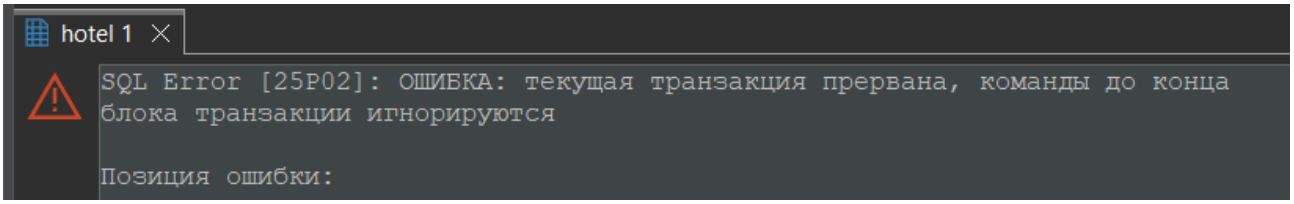

SELECT stars FROM hotel WHERE id = 10;

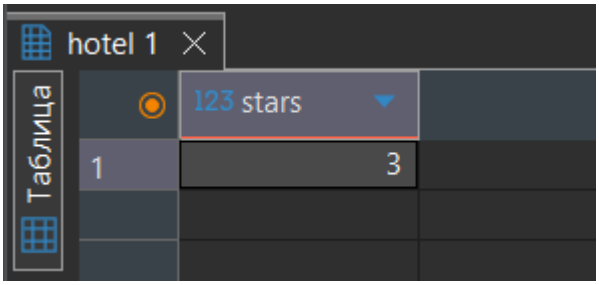
ROLLBACK;

SELECT stars FROM hotel WHERE id = 10;
```

```
SQL Error [23505]: ОШИБКА: повторяющееся значение ключа нарушает ограничение
уникальности "country_pkey"
  Подробности: Ключ "(id)=(1)" уже существует.

Позиция ошибки:
```





Задание №5: моделирование трёх аномалий параллельного доступа

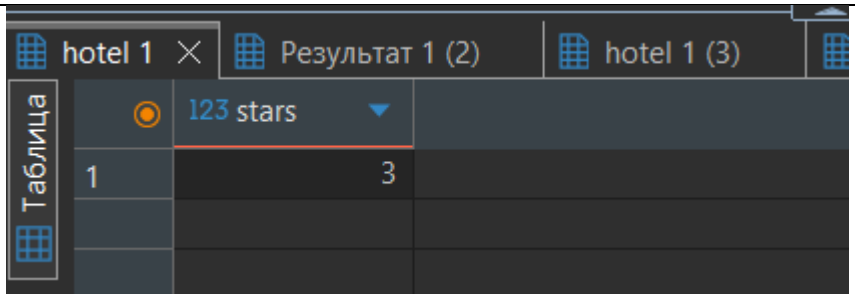
Моделирование аномалии «Неповторяемое чтение»

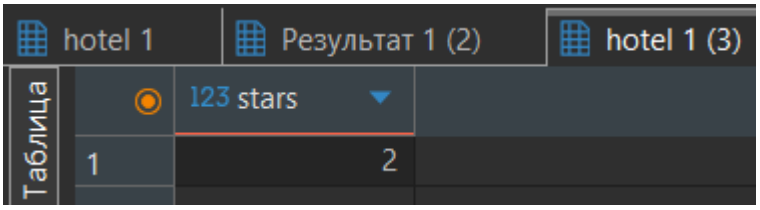
Листинг 8 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SELECT stars FROM hotel WHERE id = 10; -- Первое чтение
SELECT pg_sleep(10);
SELECT stars FROM hotel WHERE id = 10; -- Второе чтение
COMMIT;
```

Листинг 9 Скрипт Б, для выполнения во втором окне.

```
UPDATE hotel SET stars = 2 WHERE id = 10;
COMMIT;
```





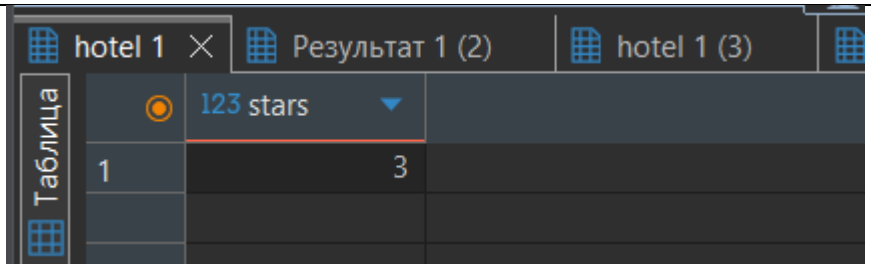
Устранение аномалии «Неповторяемое чтение»

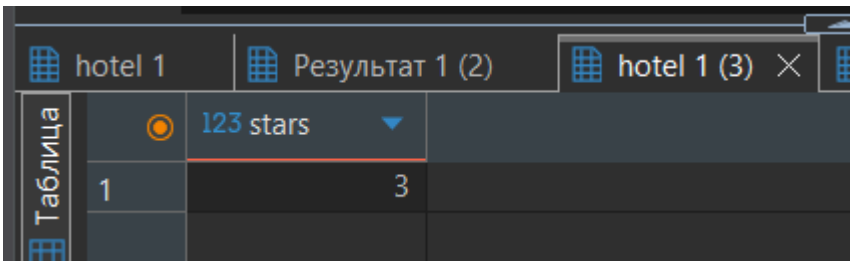
Листинг 10 Скрипт А, для выполнения в первом окне (обновлённая версия).

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT stars FROM hotel WHERE id = 10;
SELECT pg_sleep(10);
SELECT stars FROM hotel WHERE id = 10;
COMMIT;
```

Листинг 11 Скрипт Б, для выполнения во втором окне (БЕЗ ИЗМЕНЕНИЙ).

```
UPDATE hotel SET stars = 2 WHERE id = 10;
COMMIT;
```





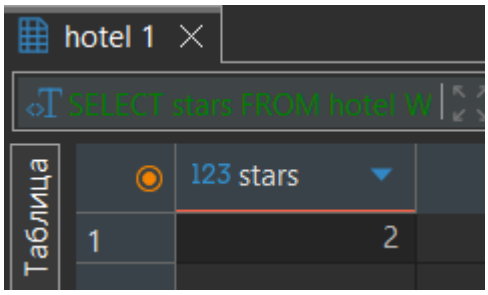
Моделирование аномалии «Грязное чтение» (Dirty Read)

Листинг 12 . Скрипт А (Окно 1)

```
BEGIN;
UPDATE hotel SET stars = 1 WHERE id = 10;
SELECT pg_sleep(10); -- Ждем 10 секунд
ROLLBACK;
```

Листинг 13 Скрипт Б (Окно 2, запускается сразу после А)

```
SELECT stars FROM hotel WHERE id = 10;
```



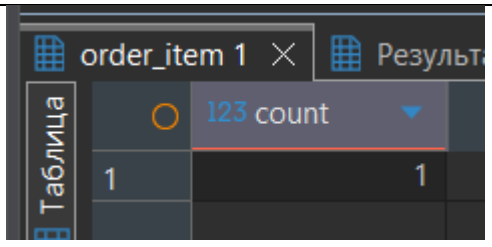
Моделирование аномалии «Фантомное чтение» (Phantom Read)

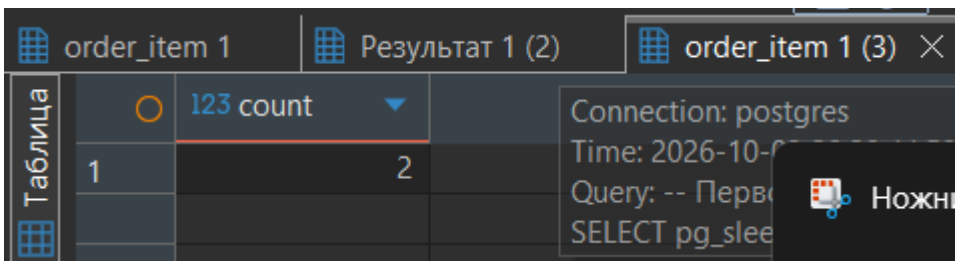
Листинг 14 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SELECT COUNT(*) FROM order_item WHERE order_id = 1; -- Первое чтение
SELECT pg_sleep(10);
SELECT COUNT(*) FROM order_item WHERE order_id = 1; -- Второе чтение
COMMIT;
```

Листинг 15 Скрипт Б, для выполнения во втором окне.

```
BEGIN;
INSERT INTO order_item (service_id, order_id, quantity, price)
VALUES (2, 1, 1, 10000);
COMMIT;
```





Устранение аномалии «Фантомное чтение» (Phantom Read)

Листинг 16 Скрипт А, для выполнения в первом окне.

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT COUNT(*) FROM order_item WHERE order_id = 1;
SELECT pg_sleep(10);
SELECT COUNT(*) FROM order_item WHERE order_id = 1;
COMMIT;
```

